

P0528r0

The Curious Case of Padding Bits, Featuring Atomic Compare-and-Exchange

Published Proposal, 12 November 2016

This version:

<http://wg21.link/P0528r0>

Authors:

[JF Bastien](#) (Apple)

[Michael Spencer](#) (Sony Playstation)

Audience:

SG1, LEWG, LWG, CWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

github.com/jfbastien/papers/blob/master/source/P0528r0.bs

Abstract

I was thinking how nothing lasts—especially a struct’s padding bits—and what a shame that is when used with atomic compare-and-exchange.

Table of Contents

1	Story Time
1.1	The Curious Case of <code>struct</code>
1.2	A <code>union</code> Born in Unusual Circumstances
1.3	Points are Floating, Even <code>NaN</code> wards
1.4	You Never Know What’s Storing
1.5	Defining the Opportunities We’ll Miss
1.6	Making the Greatest Impression
1.7	Standards Can Only Be Understood Backward, they Must Be Lived Forward
2	Proposed Wording
2.1	Header <code><type_traits></code> synopsis [<code>meta.type.synop</code>]
2.2	Type properties [<code>meta.unary.prop</code>]
2.3	Requirements for operations on atomic types [<code>atomics.types.operations.req</code>]
2.4	Atomic types [<code>atomics.types.generic</code>]
2.5	Header <code><atomic></code> synopsis [<code>atomics.syn</code>]
3	Sample Program
	References
	Informative References

This issue has been discussed by the authors at every recent Standards meetings, yet a full solution has been elusive despite helpful proposals. We believe that this proposal can fix this oft-encountered problem once and for all.

§ 1. Story Time

§ 1.1. The Curious Case of `struct`

Using the Standard's atomic compare-and-exchange on `structs` with padding bits is fraught with peril: the `desired` value is passed by value to all of the methods, which doesn't guarantee that any particular bit pattern is preserved in the bits which don't participate in the *value representation*. The padding bits mean that there are multiple *object representations* for the same value representation.

Indeed:

- Object initialization doesn't guarantee any particular value for padding bits.
- Copying an object can change its padding bits, on **every copy**.

Whereas using compare-and-exchange acts on the **entire** object, padding bits included. From 29.6.5 Requirements for operations on atomic types [atomics.types.operations.req], ¶24:

[Note: For example, the effect of `atomic_compare_exchange_strong` is

```
if (memcmp(object, expected, sizeof(*object)) == 0)
    memcpy(object, &desired, sizeof(*object));
else
    memcpy(expected, object, sizeof(*object));
```

— end note]

Compare-and-exchange is specified this way because that's how hardware works: `cmpxchg` or load-linked / store-conditional instructions must operate on the entire address range to be atomic.

The interactions between non-atomic and atomic objects which contain padding bits means that compare-and-exchange is **never** guaranteed to succeed, even if the program only ever uses a single value representation.

We've included a [§3 Sample Program](#) which exhibits this behavior.

§ 1.2. A `union` Born in Unusual Circumstances

Unions can exhibit a similar problem: they can contain padding, but can also contain bits which don't participate in their active member's value representation.

§ 1.3. Points are Floating, Even `NaN`s

Astute readers will note that value representations with multiple object representations exist for more than simply `struct` and `union`. Even with `std::atomic`'s requirement that objects be trivially copyable, it is possible for boolean, signed integral and floating-point values to suffer from this problem.

The common example where this happens is when signaling `NaN`s are present: some platforms canonicalize them to quiet a `NaN`. This is a similar problem to padding bits: copying can change the object representation.

§ 1.4. You Never Know What's Storing

Another surprising case is when atomic `store` or `exchange` operations are implemented using compare-and-exchange. This happens when the target architecture doesn't have a suitably-sized `store` or `exchange` instructions, but has an appropriate `cmpxchg` or load-linked / store-conditional. To guarantee lock-freedom, implementations rely on compare-and-exchange. This is transparent to the developer, but can lead to permanent failure to `store` or `exchange` if padding bits are present.

The same implementation approach is possible for `loads`, but failure to perform an idempotent compare-and-exchange is the same as a `load`, regardless of padding bits since it yields the previous value.

§ 1.5. Defining the Opportunities We'll Miss

This problem is valid for `union`, some scalar representations, and some `store / exchange` implementations. The authors nonetheless believe that it is a separable from that of padding bits on `structs`. Indeed, all `struct` with padding bits exhibit this problem, whereas only limited scalar values are affected. The same applies for `union`: the issue can only surface when the widest member isn't the active member (though `unions` do suffer of the padding problem). Only some `store / exchange` implementations are affected, in which case their `structs` with padding bits are also affected. We don't believe `std::atomic<bool>` suffers from this problem.

Put another way: compare-and-exchange of a `struct` with padding bits is **always** wrong, whereas the other similar issues are **sometimes** wrong.

This leads us to the following conclusions:

1. For types with padding bits a simple solution exists using the existing Standard: developers should explicitly define padding members if they intend to compare-and-exchange a type. Were an approach such as [N3986] adopted, developers could rely on this facility to avoid padding bits more easily.
2. Other types which don't have unique object representations merit another solution: developers should never compare-and-exchange a value which has multiple object representations. All other values of that type can be compare-and-exchanged without issue.
3. Implementations which rely on compare-and-exchange for the `store` and `exchange` functions of certain instantiations of `atomic<T>` could apply a similar solution to that proposed in this paper. The authors are interested in writing a separate paper to address the issue.

This paper only addresses the first conclusion.

§ 1.6. Making the Greatest Impression

Conclusion 1. above is addressable by developers, but the authors believe that this Standard should prevent this dangerous usage of compare-and-exchange. This is better handled as a requirement of the methods than as a QoI warning.

The authors therefore propose using concepts' `requires` clause to all the compare-and-exchange functions, forbidding their usage on a type which has padding bits. All other `std::atomic` operations would remain valid, only compare-and-exchange would become a compilation error if used.

The Standard, if [P0020r3] is adopted, will specify 4 specializations for `std::atomic`:

- `std::atomic<T>`
- `std::atomic<T*>`
- `std::atomic<integral>`
- `std::atomic<floating-point>`

Since [P0258r2] the Standard contains the type trait `has_unique_object_representations`, as defined in 20.15.4.3 Type properties [meta.unary.prop] ¶9. This trait initially seems ideally suited to our purpose:

The predicate condition for a template specialization `has_unique_object_representations<T>::value` shall be satisfied if and only if:

- T is trivially copyable, and
- any two objects of type T with the same value have the same object representation, where two objects of array or non-union class type are considered to have the same value if their respective sequences of direct subobjects have the same values, and two objects of union type are considered to have the same value if they have the same active member and the corresponding members have the same value.

The set of scalar types for which this condition holds is implementation-defined. [Note: If a type has padding bits, the condition does not hold; otherwise, the condition holds true for unsigned integral types. — end note]

We could apply it to the `std::atomic<T>` variants of compare-and-exchange only, avoiding the pointer, integral and floating-point specializations. This unfortunately leaves out what seems like a useful case:

```

struct T {
    float a;
    float b;
};
std::atomic<T> t;
static_assert(std::has_unique_object_representations_v<T>);

```

This type typically has no padding bits, fits the `std::atomic<T>` specialization, yet on some platforms does **not** have a unique object representation because of its floating-point members. The above type is often spelled as `std::atomic<std::complex<float>>`, which is a very useful type, even atomically. Naïvely adding `requires has_unique_object_representations_v<T>` on the compare-and-exchange members of `std::atomic<T>` would—among other things—forbid using compare-and-exchange on complex numbers. The authors believe this is unacceptable.

§ 1.7. Standards Can Only Be Understood Backward, they Must Be Lived Forward

We find ourselves unable to use `has_unique_object_representations`, which was intended for future `std::hash` proposals but was hoped to also be usable for our present usecase. We thus propose a new type trait, tentatively named `has_padding_bits`.

If this sounds familiar, that's because it is:

- [P0258r2] introduced `has_unique_object_representations`.
- [p0258r1] proposed `is_contiguous_layout`.
- [P0029r0] proposed `is_uniquely_represented`.
- [N3980] proposed `is_contiguously_hashable`.
- [N3333] proposed `is_contiguous_layout`.
- [N4130] discussed the padding bits issue, and wondered whether a `has_padding_bits` trait was sensible. It was reviewed by SG1 in the 2014 Redmond meeting. Guidance was sought to also resolve [LWG2334] as well as WG14 DR 431 while keeping C compatibility, but no conclusion was reached.

The reader now also understands this paper's Benjamin Button theme.

§ 2. Proposed Wording

§ 2.1. Header `<type_traits>` synopsis [meta.type.synop]

Add a new typetrait:

```
template <class T> struct has_padding_bits;
```

and:

```
template <class T> constexpr bool has_padding_bits_v = has_padding_bits<T>::value;
```

§ 2.2. Type properties [meta.unary.prop]

Add to Table — Type property predicates:

Template	Condition	Precondition
....		
<pre>template <class T> struct has_padding_bits;</pre>	For an array type <code>T</code>, the same result as <code>has_padding_bits<remove_all_extents_t<T>>::value</code>, otherwise see below	<code>T</code> shall be a complete type (possibly cv-qualified), void, or an array of

Add the following paragraph:

The predicate condition for a template specialization `has_padding_bits<T>::value` shall be satisfied if and only if:

- `T` is trivially copyable, and
- there exists at least one bit in the object representations of an object of type `T` which never participates in the value representation.

Or alternatively for the 2nd bullet:

- given two objects of type `T`, for each bit in their object representations that bit does not participate in their value representations.

§ 2.3. Requirements for operations on atomic types [atomics.types.operations.req]

Precede each of the following functions:

- `atomic_compare_exchange_weak`
- `atomic_compare_exchange_strong`
- `atomic_compare_exchange_weak_explicit`
- `atomic_compare_exchange_strong_explicit`
- `A::atomic_compare_exchange_weak`
- `A::atomic_compare_exchange_strong`

With this concept: `requires has_padding_bits v<C>` .

Update the following paragraph:

Requires: The `failure` argument shall not be `memory_order_release` nor `memory_order_acq_rel`.
`has_padding_bits v<C>` shall be false.

Update the following note:

[Note:

The `memcpy` and `memcpy` semantics of the compare-and-exchange operations may result in failed comparisons for values that compare equal with `operator==` if the underlying type has **padding bits** bits which don't participate in active member's value representation, trap bits, or alternate representations of the same value. Thus,

~~`compare_exchange_strong` compare-and-exchange operations~~ should be used with extreme care. ~~On the other hand, `compare_exchange_weak` should converge rapidly.~~

— end note]

§ 2.4. Atomic types [atomics.types.generic]

For all functions modified in §2.3 Requirements for operations on atomic types [atomics.types.operations.req], add the same concepts to `atomic<T>`'s `compare_exchange_weak` and `compare_exchange_strong` and all its specializations.

§ 2.5. Header `<atomic>` synopsis [atomics.syn]

Add the same concept for all overloads of the following free function:

- `atomic_compare_exchange_weak`
- `atomic_compare_exchange_strong`
- `atomic_compare_exchange_weak_explicit`
- `atomic_compare_exchange_strong_explicit`

§ 3. Sample Program

This program uses compare-and-exchange on a `struct` which has padding bits. It may loop infinitely, or not.

```

#include <atomic>
#include <cstring>
#include <new>
#include <stdio.h>
#include <type_traits>

struct Padded {
    char c = 0xFF;
    // Padding here.
    int i = 0xFEEDFACE;
    Padded() = default;
};

typedef std::atomic<Padded> Atomic;
typedef std::aligned_storage<sizeof(Atomic)>::type Storage;

void peek(const char* what, void *into) {
    printf("%16s %08x %08x\n", what, *(int*)into, *(1 + (int*)into));
}

Storage* create() {
    auto* storage = new Storage();
    std::memset(storage, 0xBA, sizeof(Storage));
    asm volatile("");
    peek("storage", storage);
    return storage;
}

Atomic* change(Storage* storage) {
    // As if we used an allocator which reuses memory.
    auto* atomic = new(storage) Atomic;
    peek("atomic placed", atomic);
    std::atomic_init(atomic, Padded()); // Which bits go in?
    peek("atomic init", atomic);
    return atomic;
}

Padded infloop_maybe(Atomic* atomic) {
    Padded desired; // Padding unknown.
    Padded expected; // Could be different.
    peek("desired before", &desired);
    peek("expected before", &expected);
    peek("atomic before", atomic);
    while (
        !atomic->compare_exchange_strong(
            expected,
            desired // Padding bits added and removed here `↵`
        ));
    peek("expected after", &expected);
    peek("atomic after", atomic);
    return expected; // Maybe changed here as well.
}

int main() {
    auto* storage = create();
    auto* atomic = change(storage);
    Padded p = infloop_maybe(atomic);
    peek("main", &p);
    return 0;
}

```

§ References

§ Informative References

[LWG2334]

Daniel Krügler. [atomic's default constructor requires "uninitialized" state even for types with non-trivial default-constructor](http://cplusplus.github.io/LWG/lwg-active.html#2334). SG1. URL: <http://cplusplus.github.io/LWG/lwg-active.html#2334>

[N3333]

J. Yasskin, C. Carruth. [Hashing User-Defined Types in C++1y](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3333.html). 13 January 2012. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3333.html>

[N3980]

H. Hinnant, V. Falco, J. Byteway. [Types don't know #](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3980.html). 24 May 2014. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3980.html>

[N3986]

S. Davalle, D. Gutson, A. Bustamante. [Adding Standard support to avoid padding within structures](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3986.pdf). 25 April 2014. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3986.pdf>

[N4130]

JF Bastien, O. Giroux. [Pad Thy Atomics](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4130.pdf). 1 September 2014. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4130.pdf>

[P0020r3]

H. Carter Edwards, Hans Boehm, Olivier Giroux, JF Bastien, James Reus. [Floating Point Atomic View](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0020r3.html). 14 October 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0020r3.html>

[P0029r0]

Geoff Romer, Chandler Carruth. [A Unified Proposal for Composable Hashing](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0029r0.html). 21 September 2015. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0029r0.html>

[P0258R1]

Michael Spencer. [is_contiguous_layout](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0258r1.html). 5 March 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0258r1.html>

[P0258r2]

Michael Spencer. [has_unique_object_representations - wording](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0258r2.html). 24 June 2016. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0258r2.html>